

Documentación Completa: Plataforma de Gestión de Proyectos

Versión: 1.0.0

Fecha de última actualización: 2025-06-19

Índice

1. [Introducción y Visión General](#)
 2. [Documentación Técnica de la Arquitectura](#)
 3. [Guía de Instalación \(Entorno de Desarrollo\)](#)
 4. [Documentación Completa de la API](#)
 - [API RESTful](#)
 - [API WebSocket](#)
 5. [Manual de Usuario](#)
 6. [Guía de Despliegue en Producción](#)
 7. [Configuración y Mantenimiento](#)
 8. [Guía de Contribución para Desarrolladores](#)
 9. [Guía de Troubleshooting](#)
 10. [Licencia y Términos de Uso](#)
-

1. Introducción y Visión General

Bienvenido a la documentación oficial de la Plataforma de Gestión de Proyectos, una solución integral, moderna y de alto rendimiento diseñada para equipos de desarrollo de software que buscan optimizar sus flujos de trabajo ágiles.

1.1. Propósito

Esta plataforma nace de la necesidad de unificar las mejores características de herramientas líderes en el mercado (como Jira, ClickUp, Trello y Mattermost) en una única aplicación cohesiva, eliminando la necesidad de saltar entre múltiples servicios. Ofrece un ecosistema completo para la planificación, ejecución, colaboración y seguimiento de proyectos de software, desde la concepción de la idea hasta el despliegue final.

1.2. Funcionalidades Principales

La plataforma es rica en funcionalidades, cubriendo todo el ciclo de vida del desarrollo de software:

- **Gestión de Proyectos y Tareas:** Creación de proyectos, tableros Kanban/Scrum, backlogs, tareas, subtareas y épicas.
- **Metodologías Ágiles:** Soporte completo para Sprints, sprint planning, burndown charts y seguimiento de la velocidad del equipo.
- **Colaboración en Tiempo Real:** Chat por canales, comentarios en tareas, menciones y notificaciones push instantáneas gracias a WebSockets.
- **Seguimiento de Tiempo:** Hojas de tiempo, cronómetros integrados en tareas y reportes de tiempo detallados.
- **Informes y Analíticas:** Dashboards personalizables con widgets para visualizar el progreso y la productividad.
- **Wiki Integrada:** Base de conocimiento por proyecto para centralizar la documentación.
- **Gestión de Usuarios y Roles:** Sistema de permisos granular para controlar el acceso a nivel de proyecto y de plataforma.

1.3. Stack Tecnológico

La plataforma está construida sobre un stack tecnológico moderno, elegido por su rendimiento, escalabilidad y robustez:

- **Frontend:** Angular 17+ con PrimeNG para una interfaz de usuario rica y reactiva, escrito en TypeScript.

- **Backend: Golang** con el framework **Gin** para una API RESTful de alto rendimiento y bajo consumo de recursos. La comunicación en tiempo real se gestiona con un hub de **WebSockets** nativo en Go.
 - **Base de Datos: PostgreSQL** como motor de base de datos relacional, aprovechando su fiabilidad y características avanzadas.
 - **Contenerización: Docker** y **Docker Compose** para una configuración de entorno consistente tanto en desarrollo como en producción.
-

2. Documentación Técnica de la Arquitectura

La plataforma se basa en una Clean Architecture con un enfoque de microservicios ligeros para facilitar la escalabilidad y el mantenimiento.

2.1. Diagrama de Contexto del Sistema (C4 - Nivel 1)

Este diagrama muestra una vista de alto nivel del sistema, sus usuarios y las interacciones con sistemas externos.

```

graph TD
    subgraph "Plataforma de Gestión de Proyectos"
        A[Sistema de Gestión de Proyectos]
    end

    subgraph Usuarios
        U1[Desarrollador]
        U2[Gestor de Proyecto]
        U3[Administrador]
        U4[Cliente/Externo]
    end

    subgraph "Sistemas Externos"
        S1[Proveedor de Identidad (SSO/LDAP)]
        S2[Repositorios Git (GitHub, GitLab)]
        S3[Herramientas CI/CD (Jenkins, etc.)]
        S4[Plataformas de Chat (Slack, Teams)]
        S5[Email Service]
    end

    U1 -- "Gestiona Tareas y Código" --> A
    U2 -- "Planifica y Supervisa" --> A
    U3 -- "Administra Usuarios y Sistema" --> A
    U4 -- "Consulta Progreso y Reporta Issues" --> A

    A -- "Autentica vía" --> S1
    A -- "Sincroniza Commits/PRs vía Webhooks" --> S2
    A -- "Recibe estado de Builds/Deploys" --> S3
    A -- "Envía Notificaciones" --> S4
    A -- "Envía Notificaciones por Email" --> S5

```

2.2. Diagrama de Contenedores (C4 - Nivel 2)

Este diagrama descompone la plataforma en sus contenedores ejecutables principales.

```
graph TD
    subgraph "Internet / Red Corporativa"
        U[Usuario (Navegador Web)]
    end

    subgraph "Entorno de Despliegue (Docker Compose / Kubernetes)"
        subgraph "Contenedores de Aplicación"
            WebApp[Frontend Nginx + Angular SPA]
            BackendApp[Backend API Golang]
        end

        subgraph "Almacenamiento de Datos"
            DB[(PostgreSQL DB)]
            Cache[(Redis)]
        end
    end

    U -- "HTTPS" --> WebApp
    WebApp -- "API REST (HTTPS)" --> BackendApp
    U -- "WebSockets (WSS)" --> BackendApp

    BackendApp -- "Lee/Escribe" --> DB
    BackendApp -- "Usa para caché y Pub/Sub" --> Cache
```

2.3. Modelo de Datos (PostgreSQL)

El esquema de la base de datos está normalizado y organizado en más de 20 tablas para soportar todas las funcionalidades. A continuación, se muestra una visión simplificada de las entidades principales:

- `users`, `roles`, `permissions`: Gestionan el acceso y la seguridad.
- `projects`, `project_members`: Definen los proyectos y quiénes pertenecen a ellos.
- `tasks`, `subtasks`, `epics`: Unidades de trabajo.
- `sprints`, `sprint_tasks`: Lógica para metodologías ágiles.
- `comments`, `attachments`: Colaboración en tareas.
- `chat_channels`, `chat_messages`: Chat integrado.
- `time_entries`, `timesheets`: Seguimiento de tiempo.
- `wiki_pages`, `wiki_versions`: Documentación interna.
- `notifications`: Sistema de alertas para los usuarios.

Las migraciones SQL completas se encuentran en `database/migrations/`.

2.4. Estructura del Proyecto

El monorepo está organizado de la siguiente manera para separar claramente las responsabilidades:

```
project-management-platform/
├─ backend/                # API REST en Golang
├─ frontend/              # SPA en Angular + PrimeNG
├─ database/              # Migraciones y seeds
├─ deployment/            # Configuración de despliegue
(Kubernetes)
├─ docs/                  # Documentación
├─ scripts/               # Scripts de utilidades (backups,
etc.)
├─ docker-compose.yml     # Orquestación para desarrollo
├─ docker-compose.prod.yml # Orquestación para producción
└─ README.md              # Este documento
```

3. Guía de Instalación (Entorno de Desarrollo)

Siga estos pasos para levantar un entorno de desarrollo local completo.

3.1. Prerrequisitos

- **Git:** Para clonar el repositorio.
- **Docker:** Versión 20.10+
- **Docker Compose:** Versión 1.29+
- **Make:** Para ejecutar comandos automatizados (opcional, pero recomendado).

3.2. Pasos de Instalación

1. Clonar el repositorio:

```
bash git clone <URL_DEL_REPOSITORIO> cd project-management-
platform
```

2. Configurar el entorno:

El backend necesita un archivo de configuración para arrancar. Copie el archivo de ejemplo:

```
bash cp backend/configs/development/config.dev.example.yml backend/
configs/development/config.dev.yml
```

Revise `config.dev.yml` y ajuste los valores si es necesario (aunque los valores por defecto están diseñados para funcionar con Docker Compose).

3. Construir e iniciar los servicios:

Utilice Docker Compose para orquestar todos los servicios (backend, frontend, base de datos, redis).

Opción A: Con Make (Recomendado)

```
bash make up
```

Este comando se encargará de construir las imágenes, levantar los contenedores y mostrar los logs.

Opción B: Con Docker Compose directamente

```
bash docker-compose -f docker-compose.yml -f docker-compose.dev.yml
up --build -d
```

4. Ejecutar migraciones y seeds de la base de datos:

Una vez que los contenedores estén en ejecución, abra otra terminal y ejecute el script para inicializar la base de datos.

Opción A: Con Make (Recomendado)

```
bash make migrate-up make seed
```

Opción B: Con Docker Compose exec

```
bash docker-compose exec backend go run ./cmd/migrate --up docker-
compose exec backend go run ./cmd/seed
```

5. Acceder a la aplicación:

¡Todo listo! Abra su navegador y visite:

- **Frontend:** `http://localhost:4200`

- **Backend API:** `http://localhost:8080`

Puede iniciar sesión con las credenciales de usuario creadas por el seeder (consulte `database/seeds/development.sql`).

3.3. Detener el entorno

Para detener todos los servicios:

Opción A: Con Make

```
make down
```

Opción B: Con Docker Compose

```
docker-compose down
```

4. Documentación Completa de la API

La plataforma expone una API RESTful para operaciones CRUD y una API de WebSocket para comunicación en tiempo real.

API RESTful

- **URL Base:** `/api/v1`
- **Autenticación:** Bearer Token (JWT) en la cabecera `Authorization`.
- **Formato de Respuesta:**

```
json { "success": true, "data": { ... }, // Contenido de la  
      respuesta "message": "Operación exitosa" }
```
- **Formato de Error:**

```
json { "success": false, "error": "código_de_error", "message":  
      "Descripción del error" }
```

Endpoints Principales (Ejemplos)

Método	Endpoint	Descripción	Rol Requerido
POST	/auth/register	Registrar un nuevo usuario.	Público
POST	/auth/login	Iniciar sesión y obtener un token JWT.	Público
GET	/projects	Obtener la lista de proyectos del usuario.	Usuario
POST	/projects	Crear un nuevo proyecto.	Manager
GET	/projects/{projectId}	Obtener detalles de un proyecto.	Miembro
GET	/projects/{projectId}/tasks	Obtener las tareas de un proyecto.	Miembro
POST	/projects/{projectId}/tasks	Crear una nueva tarea.	Miembro
PUT	/tasks/{taskId}	Actualizar una tarea.	Asignado/Manager
POST	/tasks/{taskId}/comments	Añadir un comentario a una tarea.	Miembro
GET	/sprints?project_id={projectId}	Obtener los sprints de un proyecto.	Miembro
POST	/sprints/{sprintId}/start	Iniciar un sprint.	Manager
GET	/chat/channels/{channelId}/messages	Obtener los mensajes de un canal de chat.	Miembro

(Nota: Esta es una lista simplificada. La API completa contiene más de 50 endpoints. Se recomienda generar una documentación interactiva con Swagger/OpenAPI a partir de las anotaciones del código Go).

API WebSocket

- **Endpoint de conexión:** `ws://localhost:8080/ws`

- **Autenticación:** El token JWT se envía como un parámetro en la URL de conexión: `ws://localhost:8080/ws?token=...`

Flujo de Comunicación

1. El cliente establece una conexión WebSocket.
2. El servidor valida el token y, si es exitoso, mantiene la conexión abierta.
3. El cliente no necesita suscribirse a canales explícitamente; el servidor determina qué información enviar basándose en el `userId` extraído del token.
4. El servidor empuja eventos en tiempo real al cliente cuando ocurren acciones relevantes.

Eventos del Servidor al Cliente

Evento	Payload (Ejemplo)	Descripción
<code>notification.new</code>	<pre>{ "id": "...", "message": "...", "type": "task_assigned" }</pre>	Se ha generado una nueva notificación para el usuario.
<code>task.updated</code>	<pre>{ "id": "...", "title": "...", "status": "In Progress" }</pre>	Una tarea visible para el usuario ha sido actualizada.
<code>chat.message.new</code>	<pre>{ "channelId": "...", "author": "...", "content": "..." }</pre>	Un nuevo mensaje ha sido enviado en un canal del usuario.
<code>sprint.burndown.updated</code>	<pre>{ "sprintId": "...", "data": [...] }</pre>	El gráfico burndown de un sprint activo ha cambiado.

5. Manual de Usuario

Esta sección describe las funcionalidades clave desde la perspectiva de un usuario final.

5.1. Primeros Pasos

- **Registro:** Cree una cuenta proporcionando su nombre, email y contraseña. Recibirá un email para verificar su dirección.
- **Login:** Inicie sesión con sus credenciales. Si tiene 2FA activado, se le pedirá un código de su aplicación de autenticación.
- **Dashboard Principal:** Al iniciar sesión, verá su dashboard personal. Este muestra un resumen de sus tareas asignadas, la actividad reciente en sus proyectos y widgets de métricas clave.

5.2. Gestión de Proyectos

- **Crear un Proyecto:** En el menú lateral, seleccione "Proyectos" y haga clic en "Nuevo Proyecto". Se le pedirá un nombre, una clave única (ej. "PROJ") y podrá invitar a miembros del equipo.
- **Tablero del Proyecto:** Cada proyecto tiene un tablero principal (Kanban o Scrum). Aquí puede ver las tareas organizadas por columnas (estados). Puede arrastrar y soltar tareas para cambiar su estado.

5.3. Trabajando con Tareas

- **Crear una Tarea:** En cualquier columna del tablero, haga clic en el botón "+ Nueva Tarea". Se abrirá un formulario para añadir un título, descripción (con formato Markdown), asignado, prioridad, etiquetas y fecha de vencimiento.
- **Detalle de la Tarea:** Al hacer clic en una tarea, se abre una vista detallada. Aquí puede:
 - Ver y editar todos sus campos.
 - Iniciar un cronómetro para registrar el tiempo.
 - Añadir comentarios y mencionar a compañeros (@usuario).
 - Adjuntar archivos.
 - Crear y marcar subtareas.

5.4. Colaboración

- **Chat:** Cada proyecto tiene canales de chat. Úselos para discusiones en tiempo real, compartir archivos y recibir actualizaciones. Las menciones le enviarán notificaciones directas.
 - **Wiki:** Utilice la Wiki del proyecto para documentar requisitos, guías de estilo o notas de reuniones. Las páginas son editables por todos los miembros y tienen historial de versiones.
-

6. Guía de Despliegue en Producción

Esta guía asume que se desplegará en un servidor Linux con Docker y Docker Compose.

6.1. Requisitos del Servidor

- Servidor Linux (Ubuntu 20.04+ recomendado).
- Docker y Docker Compose instalados.
- Un nombre de dominio (ej. `app.sudominio.com`).
- Un reverse proxy como Nginx o Traefik para gestionar HTTPS.

6.2. Pasos de Despliegue

1. Clonar el repositorio y entrar en el directorio.

2. Configuración de Producción:

- Cree el archivo de configuración para producción del backend:

```
bash cp backend/configs/production/config.prod.example.yml  
backend/configs/production/config.prod.yml
```
- Edite `backend/configs/production/config.prod.yml` y rellene **TODOS** los campos, especialmente las claves secretas (`jwt_secret`, etc.) y la configuración de la base de datos de producción. **NUNCA use los valores por defecto en producción.**
- Configure las variables de entorno para Docker Compose. Cree un archivo `.env` en la raíz del proyecto:

```
cp .env.example .env
```
- Edite el archivo `.env` con los datos de su base de datos de producción, contraseñas y otros secretos.

3. Iniciar los servicios:

Use el archivo `docker-compose.prod.yml`, que está optimizado para producción (no monta volúmenes de código, por ejemplo).

```
bash docker-compose -f docker-compose.yml -f docker-  
compose.prod.yml up --build -d
```

4. Configurar Reverse Proxy (Ejemplo con Nginx):

Configure un bloque de servidor en Nginx para redirigir el tráfico a los contenedores y gestionar SSL.

- Frontend (Angular) corre en el puerto 80 dentro del contenedor `frontend`.
- Backend (Go) corre en el puerto 8080 dentro del contenedor `backend`.

Un ejemplo de configuración de Nginx podría ser:

```
```nginx
```

```
server {
```

```
listen 80;
```

```
server_name app.sudominio.com;
```

```
Redirigir a HTTPS
```

```
return 301 https://<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>h</mi><mi>o</mi><mi>s</mi><mi>t</
mi></mrow></math>request_uri;
```

```
}
```

```
server {
```

```
listen 443 ssl http2;
```

```
server_name app.sudominio.com;
```

```

ssl_certificate /path/to/your/fullchain.pem;
ssl_certificate_key /path/to/your/privkey.pem;

location / {
 proxy_pass http://localhost:4200; # Asumiendo que mapeaste
 el puerto del contenedor frontend al 4200 del host
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
}

location /api/ {
 proxy_pass http://localhost:8080/api/;
 # ... mismos headers
}

location /ws {
 proxy_pass http://localhost:8080/ws;
 proxy_http_version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Connection "Upgrade";
 proxy_set_header Host $host;
}

}
...

```



## 5. Seguridad:

- Asegúrese de que los puertos de la base de datos no estén expuestos públicamente. Docker Compose los aísla por defecto en una red interna.
  - Use un firewall (como `ufw`) para permitir solo el tráfico en los puertos 80 y 443.
  - Gestione los secretos de forma segura (usando variables de entorno o un sistema como HashiCorp Vault).
- 

# 7. Configuración y Mantenimiento

## 7.1. Variables de Configuración

Toda la configuración de la aplicación se gestiona a través de archivos YAML ubicados en `backend/configs/`. El archivo específico se carga según la variable de entorno `GIN_MODE` (`development`, `testing`, `production`).

**Claves importantes en `config.*.yaml`:**

- \* `server.port`: Puerto en el que corre el backend.
- \* `database.dsn`: Cadena de conexión a PostgreSQL.
- \* `redis.addr`: Dirección del servidor Redis.
- \* `jwt.secret_key`: Clave secreta para firmar los tokens JWT. **¡Debe ser larga y aleatoria en producción!**
- \* `email.smtp_host`, `email.smtp_port`, etc.: Configuración para el envío de correos.

## 7.2. Tareas de Mantenimiento

El repositorio incluye scripts para tareas comunes en `scripts/maintenance/`.

- **Backups de la Base de Datos:**

El script `scripts/maintenance/backup.sh` realiza un volcado de la base de datos PostgreSQL. Se recomienda ejecutarlo periódicamente a través de un `cron job`.

```
bash # Ejemplo de cron job para un backup diario a las 2 AM 0 2 * *
* /path/to/project/scripts/maintenance/backup.sh
```

- **Logs:**

Puede ver los logs de los contenedores en tiempo real con:

```
bash docker-compose logs -f backend frontend
```

- **Actualización de la Aplicación:**

1. `git pull` para obtener el último código.
  2. `docker-compose -f ... build` para reconstruir las imágenes.
  3. `docker-compose -f ... up -d` para recrear los contenedores con las nuevas imágenes.
  4. Ejecutar nuevas migraciones si las hubiera: `make migrate-up`.
- 

## 8. Guía de Contribución para Desarrolladores

Agradecemos las contribuciones de la comunidad.

### 8.1. Convenciones de Código

- **Backend (Go):** Siga las convenciones de `Effective Go`. Use `gofmt` y `golint` para formatear y analizar el código.
- **Frontend (Angular):** Siga la guía de estilo oficial de Angular.

## 8.2. Flujo de Trabajo

1. Cree un `fork` del repositorio.
2. Cree una nueva rama para su funcionalidad o corrección de bug (`git checkout -b feature/nombre-feature`).
3. Implemente sus cambios.
4. **Añada pruebas unitarias** para su código.
5. Asegúrese de que todas las pruebas pasen: `make test-backend` y `make test-frontend`.
6. Envíe un `Pull Request` a la rama `main` del repositorio original.
7. En la descripción del PR, explique claramente los cambios que ha realizado.

## 8.3. Ejecución de Pruebas

- **Backend:** `make test-backend` o `docker-compose exec backend go test ./...`
  - **Frontend:** `make test-frontend` o `docker-compose exec frontend npm test`
- 

# 9. Guía de Troubleshooting

Problemas y soluciones comunes.

- **Problema:** Los contenedores no inician, error "port is already allocated".
  - **Solución:** Otro servicio en su máquina está usando uno de los puertos requeridos (ej. 4200, 8080, 5432). Detenga ese servicio o cambie los mapeos de puertos en el `docker-compose.dev.yml`.

- **Problema: El backend no puede conectar a la base de datos.**
    - **Solución:** Verifique que el nombre del host de la base de datos en `config.dev.yml` coincida con el nombre del servicio en `docker-compose.yml` (por defecto, `postgres`). Asegúrese de que las credenciales son correctas.
  - **Problema: Error de CORS en el navegador al llamar a la API.**
    - **Solución:** Asegúrese de que la configuración de CORS en `config.dev.yml` (`server.cors_allowed_origins`) incluye `http://localhost:4200`.
  - **Problema: Las migraciones fallan.**
    - **Solución:** Revise los logs del comando de migración para ver el error SQL específico. Podría ser un error de sintaxis o un problema de permisos. Puede ser necesario conectarse a la base de datos manualmente para diagnosticar. Para reiniciar desde cero, puede usar `make down-v` (¡esto borrará todos los datos!) y luego `make up`.
- 

## 10. Licencia y Términos de Uso

### 10.1. Licencia

Este proyecto se distribuye bajo la **Licencia MIT**.

Copyright (c) 2025 MiniMax Agent

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

## 10.2. Términos de Uso

- El uso de esta plataforma está sujeto a las leyes y regulaciones locales.
- El software se proporciona "tal cual", sin garantías de ningún tipo.
- Los mantenedores del proyecto no se hacen responsables de ninguna pérdida de datos o daño resultante del uso (o mal uso) de este software. Se recomienda encarecidamente realizar backups regulares.